

```

#from skopt import BayesSearchCV
from sklearn.model_selection import StratifiedKFold
from sklearn.utils.estimator_checks import parametrize_with_checks

import serializer
from typing import List

def train_recommendedparams(X_train, y_train, algorithms: List[str]):
    recommended_params = []
    # Comprobación del tamaño del conjunto de datos
    n_samples, n_features = X_train.shape
    if n_samples > 10000 or n_features > 50:
        # Reducción del espacio de búsqueda y número de iteraciones
        n_iter = 25
        param_factor = 0.5
    else:
        n_iter = 50
        param_factor = 1.0

    for algorithm in algorithms:
        model_class = serializer.model_classes[algorithm]
        # Estimación de parámetros iniciales
        estimator, _ = parametrize_with_checks(model_class)
        estimator.fit(X_train, y_train)
        initial_params = estimator.get_params()

        # Definir el espacio de búsqueda para BayesSearchCV
        param_distributions = get_default_params(model_class, X_train, y_train, param_factor)

        # Realizar la búsqueda bayesiana de hiperparámetros
        opt = BayesSearchCV(model_class(), param_distributions, n_iter=n_iter,
cv=StratifiedKFold(n_splits=5), random_state=42)
        opt.fit(X_train, y_train)
        best_params = opt.best_params_
        recommended_params.append({'name': algorithm, 'initial_params': initial_params,
'params': best_params})
    return recommended_params

def get_default_params(model_class, X_train, y_train, param_factor):
    default_params = {}
    # Utilizamos parametrize_with_checks para estimar los valores de los parámetros iniciales
    estimator, _ = parametrize_with_checks(model_class)
    estimator.fit(X_train, y_train)
    params_dict = estimator.get_params()
    for param, value in params_dict.items():
        if isinstance(value, int) or isinstance(value, float):
            # Si el parámetro es un número, definimos un rango de búsqueda en función del valor
estimado y el factor de reducción
            default_params[param] = (value * param_factor / 10, value * param_factor * 10) #
Definimos un rango de búsqueda
        elif isinstance(value, str):
            # Si el parámetro es una cadena, dejamos que el modelo elija entre algunos valores
predefinidos
            default_params[param] = [value] # Definimos una lista de valores posibles
        else:
            # Otros tipos de parámetros (como booleanos) pueden no necesitar optimización
            pass
    return default_params

```